

Lecture 13: Indirect Communication, Part 2

COMP2014 Distributed Systems

Chris Greenhalgh

School of Computer Science

Contents

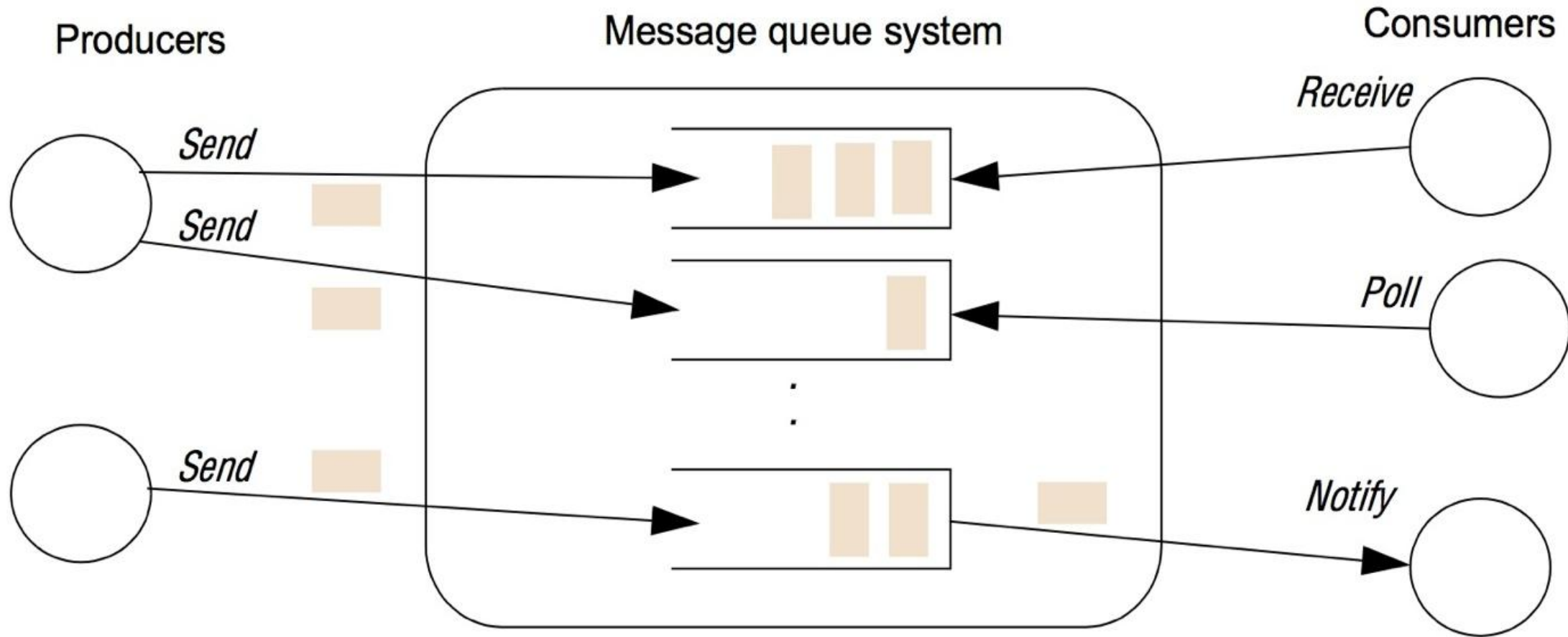
- Message queues
 - Example: JMS
- Distributed Shared Memory
- Tuple spaces
 - Example: JavaSpaces
 - Example: EQUIP2
- Example: REDIS

Message Queues (Section)

Message Queues Introduction

- Message queues (MQ) support (mainly) one-to-one communication
 - Cf. one-to-many of group communication & pub-sub
 - (Although many also have one-to-many as an option)
- Space uncoupled – **messages** are sent via message **queues**
 - Specific **producer/consumer** not known
- Time uncoupled – message queues exist independently of message producers and consumers
 - So messages can be persisted in queues

The message queue paradigm



Instructor's Guide for Coulouris, Dollimore and Kindberg Distributed Systems: Concepts and Design Edn. 4
© Pearson Education 2005

Message queue operations

- **Send** – add a message to a queue
 - Typically to the tail of the queue, but may be configurable
- **Receive** – take a message from a queue
 - Typically the head of the queue, but may be configurable or queue may support priorities
- **Poll** – check head of queue without taking the message
- **Notify** – inform a potential consumer that a message is available
 - So that a consumer doesn't need to poll

Message Queue Applications

- Commonly used for integrating back-office systems
 - E.g. e-commerce site, business process management
- Also known as message-oriented middleware (MOM)
- E.g.
 - IBM MQ series, <https://www.ibm.com/products/mq>
 - <https://www.rabbitmq.com/>
 - Java Messaging Service (JMS) [see later slide]
 - Redis [see later slides]
 - Azure Service Bus “combines enterprise messaging capabilities with rich publish-subscribe semantics”
 - <https://azure.microsoft.com/en-gb/products/service-bus/>

Implementation notes

- **Message** typically comprises:
 - **Destination**, identifying the queue
 - **Metadata** or header
 - E.g. priority, delivery mode
 - **Body**, i.e. payload or content of message
- Message queues are typically **persistent** and **reliable**
 - i.e. every message is eventually received (validity) and is uncorrupted and not duplicated (integrity)
 - However there is no guarantee of **timeliness**
 - In many enterprise systems MQ operations may be included in **transactions** together with database operations

Example: Java Messaging Service (JMS)

- Java Messaging Service (JMS) is a common API specification Java Message Queue systems
 - JMS **client** is producer and/or consumer
 - JMS **provider** implements the MQs
 - JMS **message** has: header (including destination), properties, body
 - JMS also supports pub-sub (one-to-many) semantics if messages are sent to **topics** rather than **queues**

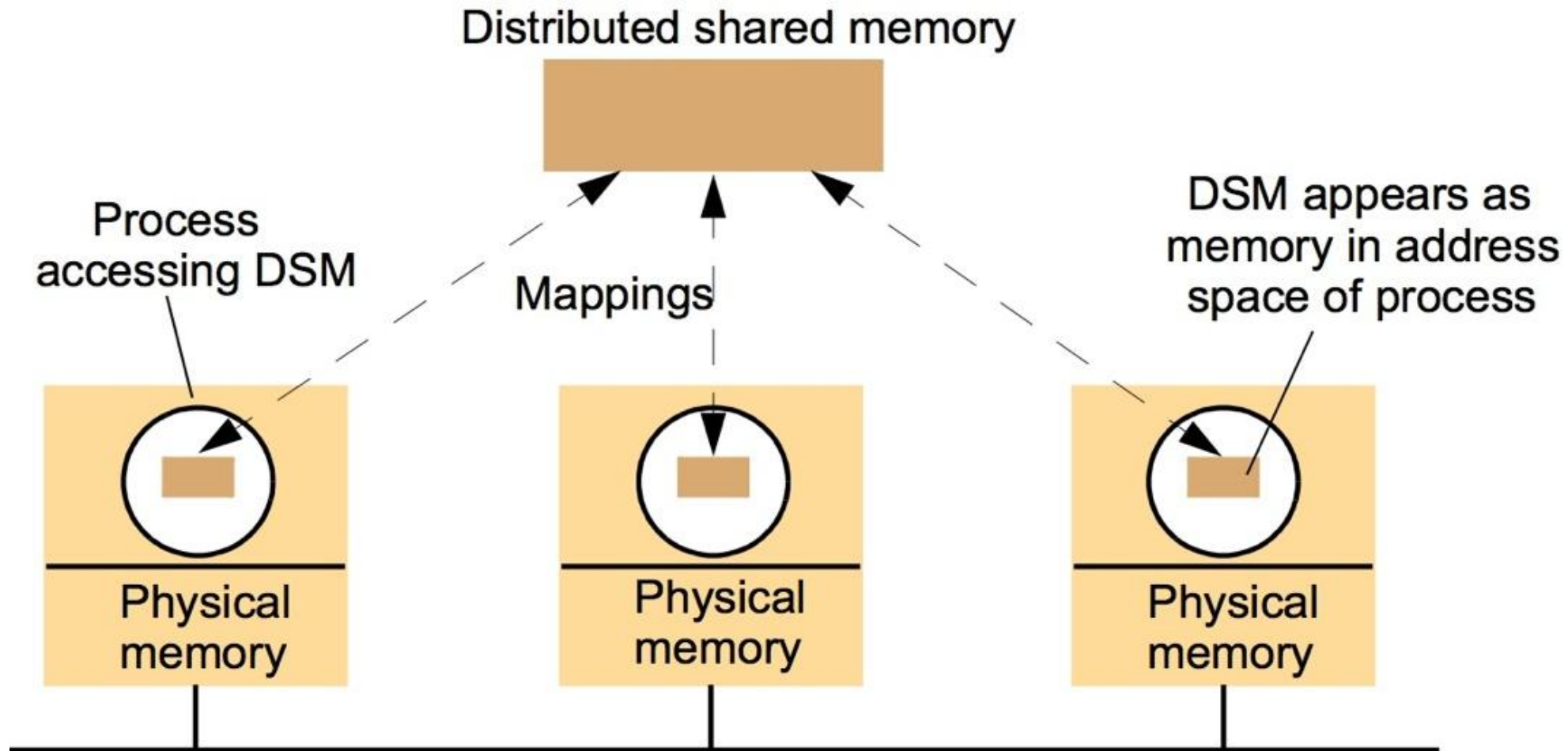
<https://www.oracle.com/java/technologies/java-message-service.html>

Distributed Shared Memory (section)

Distributed shared memory

- **Distributed shared memory (DSM)** systems emulate shared memory between processes on different computers
 - i.e. program code in different processes can communicate/coordinate by reading/writing memory locations in a single global address space
- The DSM system sends messages between nodes to maintain a consistent view of memory
- Much slower than local shared memory
- Typically requires additional disciplines such as use of locks
 - which signal to the DSM system that access will or has occurred

The distributed shared memory abstraction



Q: anyone heard of
tuple spaces?

Tuple Spaces

Tuple space communication

- First introduced by David Gelernter (Yale University, 1985)
 - Termed “generative communication”
- Several processes can share a single **tuple space**
 - Similar to distributed shared memory
 - Also known as a “shared blackboard” in AI
- All values are **tuples**
 - i.e. sequences of values such as <“Fred”, 1985>
- Tuples can be written to (**write**), read from (**read**) or taken from (**take** = read and delete) the tuple space
- Reads and takes are done by **pattern matching**
 - E.g. <String, 1985> would match <“Fred”, 1985> (returns one match)
 - Note: unlike DSM, tuples do NOT have addresses

The tuple space abstraction



Characteristics

- Like message queues, tuple spaces are both space uncoupled and time uncoupled
- (many) Tuple spaces can also support **transactions**
 - I.e. to atomically group a sequence of read/take/write operations
 - See later lecture on reliability & failure
 - E.g.
 - update a tuples by (1) create transaction (2) take old value (3) write new value (4) commit transaction
 - read ALL tuples matching a pattern by (1) create transaction, (2) repeatedly take a matching tuple until none are left, then (3) abort the transaction, so all the tuples are returned

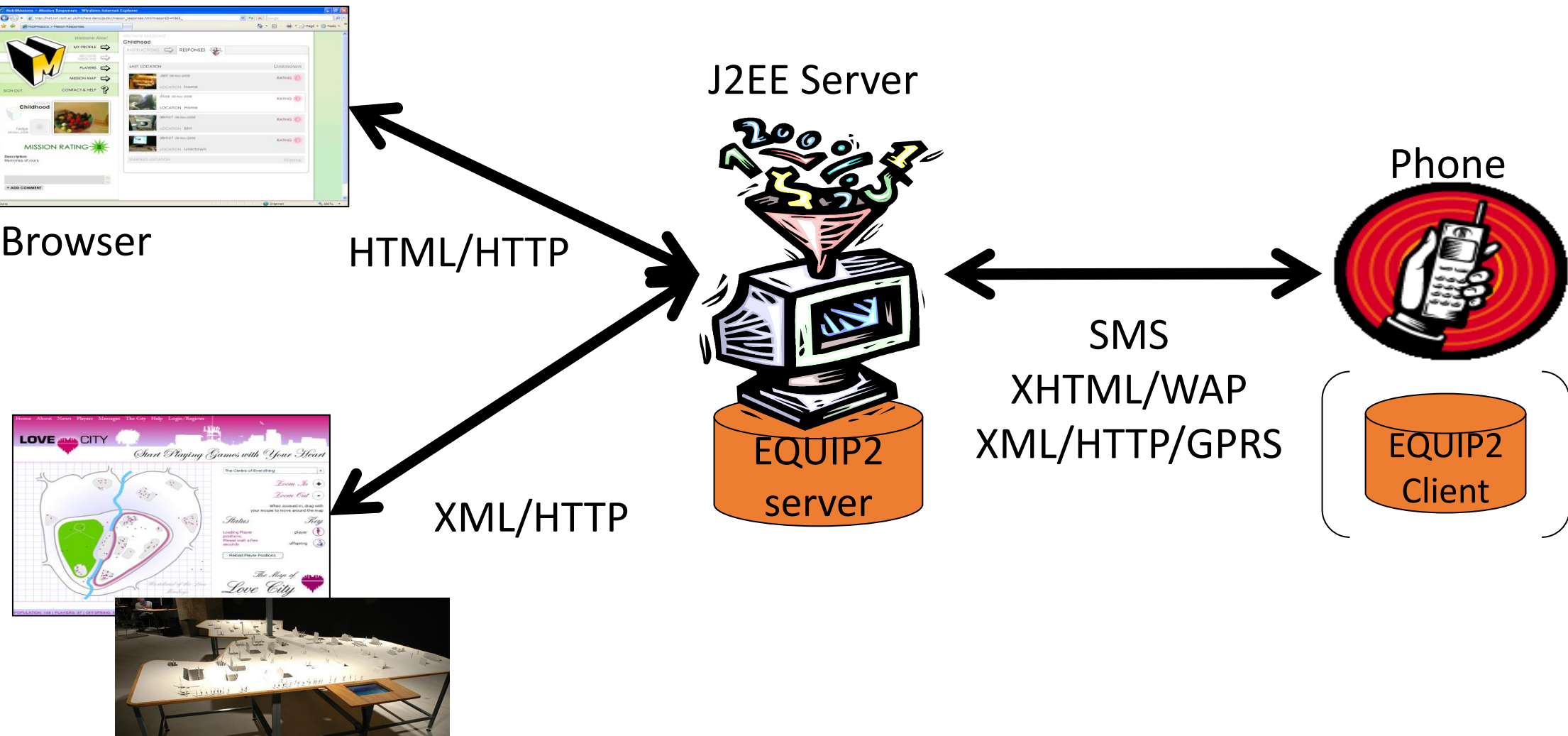
Example: JavaSpaces

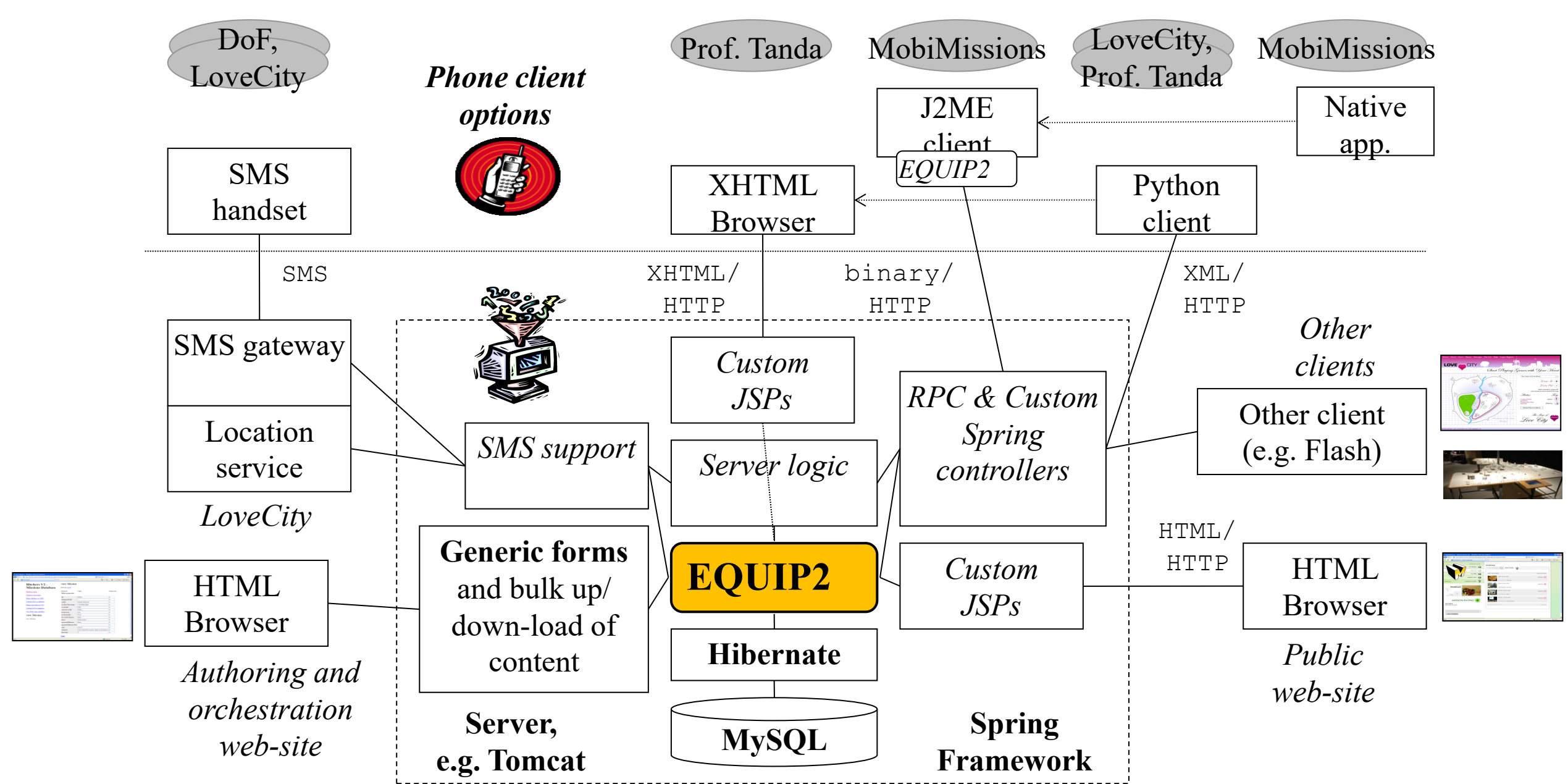
- **JavaSpaces** is a tool for tuple space communication developed by Sun as part of JINI
 - Strictly it is an **object space**
 - i.e. the shared space holds objects, sub-classes of `net.jini.core.entry.Entry`
 - <https://en.wikipedia.org/wiki/Jini>
 - Then became Apache River <http://river.apache.org/> - now retired

Example: EQUIP2 (Not examinable)

- A library supporting **dataspaces**
 - Open source - <http://equip.sourceforge.net/>
 - NB not maintained since 2007 ☺
 - Holds objects, i.e. object space
 - More operations than a minimal tuple space
 - Synchronous operations: **match**, get, add, **update**, remove
 - Asynchronous notifications: any change, pattern-matched changes
- Distributed implementation and caching
 - Support for intermittent and low-bandwidth connections
- Supports loosely coupled applications
 - input devices publish to dataspaces
 - “business”/application logic triggered by changes in dataspaces
 - Communication performed by copying/moving between dataspaces

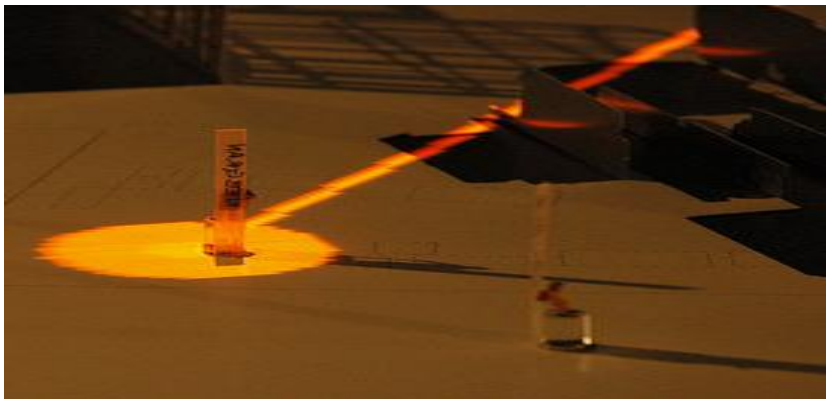
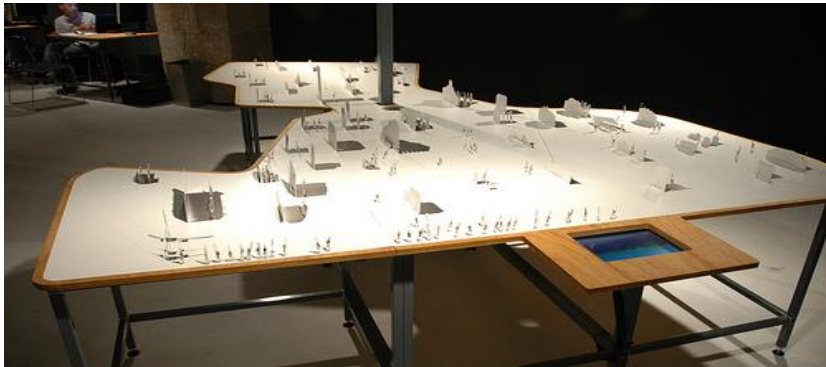
Common platform/toolkit for mobile phone-based experiences





Day of the Figurines

- SMS, web and public installation



RECEIVED

09:57pm, you've arrived at the Trafalgar Sq, Italian football is on the telly. BERNARD, EVE and SUCHDA are here. There are many PINTS here.

SENT

Use pint

RECEIVED

09:57pm, you down the pint of Vale leaving a modest foam moustache on your top lip. You are feeling well.

MobiMissions

- J2ME MIDlet with camera support plus web



MobiMissions > Home - Windows Internet Explorer

http://holt.mrl.nott.ac.uk/hitchers-demo/public/home.htm

MobiMissions > Home

Welcome Alice!

MY PROFILE →

BROWSE MISSIONS →

PLAYERS →

MISSION MAP →

SIGN OUT

CONTACT & HELP ?

NEWEST MISSION

MISSION find this

demo1 23-Aug-2007

Where exactly is this?

MORE →

HOME

Top Scoring Players

julia	hannah	Alice	hadge
SCORE 160	SCORE 86	SCORE 68	SCORE 46

Top Missions

	Stinky Alice 20-Oct-2006 LAST LOCATION Home	MISSION RATING 8 RESPONSES 3
	Childhood hadge 06-Nov-2006 LAST LOCATION On player demo1's phone	MISSION RATING 8 RESPONSES 4
	Julia Alice 20-Oct-2006 LAST LOCATION Home	MISSION RATING 6 RESPONSES 2
	Make a change mobi 16-Oct-2006 LAST LOCATION On player mobi's phone	MISSION RATING 5 RESPONSES 2

COMP2014 Lecture 13: Indirect Communication 2

EQUIP2 discussion

Some architectural choices & patterns...

- Dataspace / tuple-space approach
 - Indirect communication: space & time uncoupled
 - Pattern matching => Loosely coupled
 - Integrated notifications ~ publish-subscribe
- Main dataspace used as integration hub

Example: Redis

Note, the details of Redis are NOT directly examined, but it is a useful example to know something about and to experiment with (e.g. in Lab 7).

Introducing Redis

<https://redis.io/>

- Redis
 - Is an in-memory **data structure** store,
 - plus (simple) publish-subscribe service
 - used as a database, cache, and message broker.
 - But note persistence limitations wrt database!
- Provides data structures
 - E.g. strings, hashes, lists, sets, sorted sets with range queries, bitmaps, hyperloglogs, geospatial indexes, and streams (see next slides)
- Each data-structure is identified by a key (=unique name or ID)
 - Includes operations to list keys and key ranges
- Each data type has different set of operations...

Example Redis data structures / uses

<https://redis.io/topics/data-types-intro>

- String – basic key-value store/cache
 - Various commands: SET “key” “value” [EX time], GET “key”, ...
 - <https://redis.io/commands/?group=string>
 - Note, any key can be set to expire (disappear) after a time
- List – can be used like a message queue
 - Add and remove items at either end of the list
- Hash – i.e. set of name-value pairs, can be used like a row in a table
 - Get set all or specific hash entries
- Stream – a mix of message queue & pub-sub,
 - like an Apache Kafka stream, <https://kafka.apache.org/>

Redis Notes

Q: Why define the protocol directly over TCP and not HTTP? Pros & cons?

- In-memory => fast but not completely durable
 - Some updates might be lost after a crash
- Standard TCP protocol
 - Client library implementations in LOTS of languages
 - <https://redis.io/docs/reference/protocol-spec/>
 - <https://redis.io/clients>
- Missing standard RDBMS functions
 - No general transactions
 - No secondary indexes or query optimisation
- For particular applications...
 - You have to choose the right data structure
 - And use it the right way
- Host it yourself, or use cloud-hosted service <https://redis.io/cloud/>

Message Queue Summary

- Message queues (MQ) support **one-to-one** communication via named **message queues**
 - Space and time uncoupled
 - Message queues are (typically) persistent
 - Operations: **send, receive, poll, notify**
 - Can be included in transactions
- For example:
 - IBM MQ - <https://www.ibm.com/products/mq>
 - <https://www.rabbitmq.com/>
 - Redis

DSM & Tuple Spaces Summary

- **Distributed Shared Memory** emulates a global shared address space across multiple nodes/processes
- **Tuple spaces** support indirect communication through shared spaces holding **tuples**
 - Operations: **write, read, take, notify**
 - Write/take allows **one-to-one** indirect communication like message queues
 - Write/Notify/read allow pub-sub style **one-to-many** communication
 - Read and take use **pattern matching**
 - Like pub-sub content filtering
 - Tuples are **persistent**
 - Unlike pub-sub events

Next steps

- Lab 7: Redis
- Lecture 14: Q&A, Revision
 - Then on to Peer-to-Peer, Multicast...